

PYTHON 3.12+ CHEAT SHEET

Practical syntax, idioms, and standard-library patterns for everyday programming

1. Python Basics

```
# comment; names bind to objects
x, name, active = 42, "Ada", True
a = b = 0                # multiple assignment
missing = None           # absence of a value
```

Type	Typical literal / conversion
int, float, complex	12, 3.5, 2+3j; int("12"), float(x)
bool, NoneType	True, False, None; bool(x)
str, bytes	"text", b"raw"; str(x), bytes(s, "utf-8")
list, tuple	[1,2], (1,2); mutable / immutable sequence
dict, set	{"a":1}, {1,2}; mapping / unique values

```
type(x) is int           # exact type
isinstance(x, (int, float)) # type or subclass
a == b                   # equal values
a is b                   # identical object
                           (use: x is None)
```

Falsy: False, None, numeric zero, and empty strings/containers. Everything else is truthy.

2. Strings

Strings are immutable Unicode sequences; operations return new strings.

```
s = "Python"
s[0], s[-1]      # ('P', 'n')
s[1:4], s[::-1]  # ('yth', 'nohtyP')
len(s), "th" in s # (6, True)
```

Expression	Result / purpose
s.split(",")	split into a list
"-".join(parts)	join strings with separator
s.strip()	trim surrounding whitespace
s.replace("a","b")	replace all occurrences
s.find("th")	first index (2), or -1
s.startswith("Py")	prefix test
s.endswith(".py")	suffix test
s.upper(), s.lower()	change case
"x" in s	substring membership

```
name, age = "Alice", 30
f"{name} is {age} years old"
f"{1234.5:,.2f}"           # '1,234.50'
f"{0.875:.1%}"             # '87.5%'
```

Alignment and display: f"{x:>10}" (right), f"{x:<10}" (left), f"{x:^10}" (center), f"{n:08d}" (zero-pad), f"{x=!r}" (debug).

```
message = """A multiline string
may span several lines."""
raw_path = r"C:\new\data"    # backslashes stay literal
```

3. Numbers, Math, and Bits

Syntax	Meaning
a+b, a-b, a*b, a/b	arithmetic; / returns float
a//b, a%b, a**b	floor division, remainder, power
round(x, 2), abs(x)	round; magnitude
min(xs), max(xs), sum(xs)	aggregate values

```
import math
math.sqrt(9); math.ceil(2.1); math.floor(2.9)
math.isclose(a, b, rel_tol=1e-9)
math.pi, math.e, math.log(x), math.sin(angle)
```

// and % round toward negative infinity: -7 // 2 is -4 and -7 % 2 is 1. divmod(a, b) returns both at once.

Bitwise operators

These act on the binary bits of an int. Python integers are signed and unbounded, so they behave like two's complement with infinite sign extension.

Syntax	Meaning
a & b	AND: 1 where both bits are 1
a b	OR: 1 where either bit is 1
a ^ b	XOR: 1 where the bits differ
~a	NOT: invert every bit; equals -a - 1
a << n	shift left; same as a * 2**n
a >> n	shift right; same as a // 2**n

```
x = 0b1010           # 10; also 0o12, 0xA, 1_000
bin(10), hex(255)    # ('0b1010', '0xff')
int("1010", 2)       # 10
10 & 6, 10 | 6, 10 ^ 6 # (2, 14, 12)
~10, 10 << 1, 10 >> 1  # (-11, 20, 5)
(10).bit_length(), (10).bit_count() # (4, 2)
```

```
mask = 1 << i        # a 1 in position i only
n | mask              # set bit i
n & ~mask              # clear bit i
n ^ mask              # toggle bit i
(n >> i) & 1           # read bit i -> 0 or 1
n & (n - 1)           # clear the lowest set bit
n & -n                 # isolate the lowest set bit
n > 0 and n & (n - 1) == 0 # exact power of two?
```

Bitwise operators bind tighter than comparisons but looser than arithmetic, so `n & 1 == 0` groups as `(n & 1) == 0`. `>>` preserves the sign and there is no unsigned `>>>`; mask with `& 0xFF` to emulate a fixed width. For named flags, prefer `enum.IntFlag` over bare constants.

4. Lists

Mutable ordered sequences; indexing and slicing match strings.

```
items = ["a", "b", "c"]
items[0], items[-1], items[1:]
items[1] = "B"; "a" in items
empty = []; shallow = items.copy() # shallow copy
```

Mutation	Effect
<code>xs.append(x)</code>	add one item at end
<code>xs.extend(more)</code>	append all items
<code>xs.insert(i,x)</code>	insert before index <code>i</code>
<code>xs.remove(x)</code>	delete first <code>x</code> ; raises if absent
<code>x = xs.pop(i)</code>	delete and return item (last by default)
<code>xs.clear()</code>	remove every item
<code>xs.reverse()</code>	reverse in place
<code>xs.sort(key=..., reverse=True)</code>	sort in place

```
nums = [4, 1, 3, 2]
ordered = sorted(nums)           # new list
nums.sort(reverse=True)          # mutate nums
even_squares = [x*x for x in nums if x % 2 == 0]
```

Useful slices: `xs[:n]` first `n`; `xs[n:]` rest; `xs[::2]` every other; `xs[::-1]` reversed copy; `del xs[i:j]` delete range.

5. Tuples and Sets

Tuples

Immutable sequences are ideal for fixed records and unpacking. A one-item tuple needs a comma.

```
point = (10, 20); single = (10,)
x, y = point
first, *middle, last = values
def bounds(xs): return min(xs), max(xs)
```

Sets

Mutable collections of unique hashable values; no positional indexing.

```
a, b = {1, 2, 3}, {3, 4}
a.add(5); a.discard(9)      # no error if absent
a.remove(2)                 # KeyError if absent
a | b, a & b, a - b          # union, intersection,
                             # difference
```

`x` in `a` tests membership; `set(items)` removes duplicates (order not preserved). Use `frozenset(...)` for an immutable, hashable set.

6. Dictionaries

Mappings preserve insertion order and require hashable keys.

```
user = {"name": "Alice", "age": 30}
name = user["name"]         # KeyError if absent
age = user.get("age", 0)    # safe default
user["active"] = True       # add / update
user.update(age=31, city="Paris")
```

Expression	Purpose
<code>k in d</code>	key membership
<code>d.keys(), d.values()</code>	dynamic views
<code>d.items()</code>	(key, value) pairs
<code>d.pop(k, default)</code>	remove key and return value
<code>del d[k]</code>	remove key; raises if absent
<code>d.setdefault(k, v)</code>	get existing or insert default
<code>d1 d2</code>	new merged dict; right side wins

```
for key, value in user.items():
    print(key, value)
lengths = {word: len(word) for word in words}
merged = defaults | overrides
```

Counting and grouped values

```
from collections import Counter, defaultdict
counts = Counter(words)           #
counts.most_common(3)
groups = defaultdict(list)
for item in rows: groups[item.key].append(item)
```

7. Conditions

```
if score >= 90:
    grade = "A"
elif score >= 80:
    grade = "B"
else:
    grade = "C"
```

Comparisons: `==` `!=` `<` `<=` `>` `>=`; logic: `and` `or` `not`. Comparisons chain naturally: `0 <= x < 10`. Short-circuit logic can supply defaults: `name = supplied or "Guest"`.

```
status = "adult" if age >= 18 else "minor"
if ready and not cancelled:
    run()
```

8. Loops

```
for item in items:
    process(item)
for i in range(0, 10, 2):           # 0, 2, ..., 8
    print(i)
```

```
for i, value in enumerate(items, start=1):
    print(i, value)
for name, score in zip(names, scores, strict=True):
    print(name, score)
```

```
for key, value in data.items():
    if value is None: continue
    if key == target: break
else:
    print("no break / not found")
```

```
attempts = 3
while attempts > 0:
    if try_once(): break
    attempts -= 1
```

`break` exits the nearest loop; `continue` skips to its next iteration. Loop else runs only when no `break` occurs.

9. Functions

```
def greet(name: str, greeting: str = "Hello") -> str:
    """Return a short greeting."""
    return f"{greeting}, {name}!"

greet("Ada", greeting="Hi")
```

Parameters may be positional or keyword. `/` marks positional-only; `*` marks following parameters keyword-only.

```
def clamp(x, /, low=0, high=100, *, strict=False):
    if strict and not low <= x <= high:
        raise ValueError(x)
    return max(low, min(x, high))
```

```
def collect(*args, **kwargs):
    return args, kwargs           # tuple, dict
```

```
nums = [3, 7]
range(*nums)                     # unpack positional
args
opts = {"sep": " - ", "end": "!\n"}
print("a", "b", **opts)         # unpack keyword
args
```

Lambdas are short expressions, best used as keys/callbacks: `sorted(users, key=lambda u: u["age"])`.

Mutable default trap: defaults are created once.

```
def add(x, items=None):
    items = [] if items is None else items
    items.append(x)
    return items
```

10. Comprehensions and Generators

Result	Pattern
list	<code>[f(x) for x in xs if test(x)]</code>
set	<code>{f(x) for x in xs}</code>
dict	<code>{k: f(k) for k in keys}</code>
generator	<code>(f(x) for x in xs)</code> (lazy)

```
even_squares = [x*x for x in nums if x % 2 == 0]
by_id = {u.id: u for u in users}
total = sum(x*x for x in nums)  # no intermediate list
flat = [x for row in matrix for x in row]
```

Prefer a normal loop when the expression needs side effects or becomes hard to scan.

11. Most Useful Built-ins

Call	Use
<code>len(x)</code>	number of items
<code>range(stop)</code>	integer sequence (lazy)
<code>enumerate(xs)</code>	(index, value) pairs
<code>zip(a,b)</code>	pair corresponding items
<code>sorted(xs,key=...)</code>	new sorted list
<code>reversed(xs)</code>	reverse iterator
<code>any(tests) / all(tests)</code>	some / every item truthy
<code>sum(xs)</code>	numeric total
<code>min(xs), max(xs)</code>	extrema; accept key=
<code>next(it, default)</code>	next item or default

```
oldest = max(users, key=lambda u: u.age)
has_errors = any(row.error for row in rows)
all_valid = all(validate(x) for x in items)
first = next((x for x in xs if test(x)), None)
```

`map(f, xs)` and `filter(test, xs)` return iterators; comprehensions are usually clearer.

12. Exceptions

Catch only errors you can handle; prefer specific exception types.

```
try:
    value = int(text)
except ValueError as exc:
    value = 0
else:
    use(value)  # conversion succeeded
finally:
    cleanup()  # always runs

if amount < 0:
    raise ValueError("amount must be nonnegative")
```

Multiple types: `except (TypeError, ValueError):`. Re-raise the current exception with bare `raise`; preserve cause with `raise NewError(...) from exc`.

13. Files and pathlib

Context managers close files even when an error occurs. Specify text encoding.

```
with open("data.txt", "r", encoding="utf-8") as f:
    text = f.read()  # whole file
with open("out.txt", "w", encoding="utf-8") as f:
    f.write(text)
```

"a" appends; "b" selects binary data (for example "rb"). Iterate lines without loading the whole file:

```
with open("data.txt", encoding="utf-8") as f:
    for line in f:
        process(line.rstrip("\n"))
```

Prefer pathlib for paths

```
from pathlib import Path
p = Path("data") / "input.txt"
text = p.read_text(encoding="utf-8")
Path("out.txt").write_text("Hello",
    encoding="utf-8")
```

```
p.exists(); p.is_file(); p.name; p.suffix; p.parent
p.mkdir(parents=True, exist_ok=True)
for file in Path(".").glob("*.py"):
    print(file)
```

Use `Path.rglob("*.csv")` recursively; `p.rename(new)` moves/renames; `p.unlink(missing_ok=True)` deletes a file.

14. Modules and Imports

```
import statistics
import numpy as np
from pathlib import Path
from package.module import useful_name
```

Imports run module top-level code once and bind names. Put script entry behavior behind a guard:

```
def main() -> None:
    ...

if __name__ == "__main__":
    main()
```

15. Classes and Data Classes

```
class Account:
    def __init__(self, owner: str, balance=0.0):
        self.owner, self.balance = owner, balance

    def deposit(self, amount: float) -> None:
        self.balance += amount

class Savings(Account):
    def __init__(self, owner, rate):
        super().__init__(owner)
        self.rate = rate

    @property
    def interest(self): return self.balance * self.rate
```

Use data classes for data containers; they generate `__init__`, equality, and a useful representation.

```
from dataclasses import dataclass, field
```

```
@dataclass(slots=True)
class User:
    name: str
    tags: list[str] = field(default_factory=list)
```

16. Iterators and Generators

An iterable supplies an iterator; `next()` pulls one value. Generators produce lazy streams and retain local state.

```
it = iter([10, 20])
next(it), next(it)  # 10, 20
next(it, None)      # None (default avoids StopIteration)
```

```
def countdown(n):
    while n:
        yield n
        n -= 1
```

```
squares = (x*x for x in range(1_000_000))
```

Generators are single-use; materialize with `list(gen)` only when needed.

17. Standard Library Essentials

collections

```
from collections import Counter, defaultdict, deque
Counter("banana").most_common(2)  # [('a', 3),
                                   ('n', 2)]
groups = defaultdict(list)
queue = deque([1, 2]); queue.append(3);
queue.popleft()
```

heapq (min-heap / priority queue)

A heap is an ordinary list kept in heap order, so `h[0]` is always the smallest item. Push and pop cost $O(\log n)$ and heapify costs $O(n)$; the list as a whole is *not* sorted.

```
import heapq
h = [5, 1, 3]
heapq.heapify(h)           # -> [1, 5, 3],
                           rearranged in place
heapq.heappush(h, 2)
smallest = heapq.heappop(h) # 1; peek instead
                           with h[0]
heapq.heappushpop(h, 7)    # push then pop, in one
                           pass
heapq.heapreplace(h, 7)    # pop then push; heap
                           must be non-empty

heapq.nsmallest(3, xs)      # sorted;
                           cheap for small k
heapq.nlargest(3, words, key=len)
list(heapq.merge(sorted_a, sorted_b)) # lazy merge
                                       of sorted inputs
```

There is no max-heap, so negate the values (or the sort key):

```
maxh = [-x for x in xs]; heapq.heapify(maxh)
largest = -heapq.heappop(maxh)
```

Entries are compared as whole tuples, so insert a unique tie-breaker ahead of any payload that is not orderable:

```
from itertools import count
tie = count()           # unique, increasing
heapq.heappush(pq, (priority, next(tie), task))
priority, _, task = heapq.heappop(pq) # lowest
                                       priority first
```

Use `queue.PriorityQueue` instead when several threads share the queue.

datetime

```
from datetime import datetime, date, timedelta,
timezone
now = datetime.now(timezone.utc); today =
date.today()
dt = datetime.strptime("2026-08-08", "%Y-%m-%d")
dt.strftime("%d %b %Y"); tomorrow = today +
timedelta(days=1)
```

Use timezone-aware datetimes for stored/shared timestamps.

json

```
import json
data = json.loads('{"ok": true}')
text = json.dumps(data, indent=2,
ensure_ascii=False)
data = json.loads(Path("data.json").read_text())
Path("out.json").write_text(json.dumps(data,
indent=2))
```

random

```
import random
n = random.randint(1, 6)           # inclusive endpoints
item = random.choice(items)
random.shuffle(items)              # in place
```

Use `secrets` rather than `random` for passwords, tokens, or security.

Regular expressions

Use raw strings for patterns; compile a pattern when reusing it often.

```
import re
m = re.search(r"\b\d+\b", text)      # first match /
None
nums = re.findall(r"\d+", text)      # all matches
clean = re.sub(r"\s+", " ", text).strip()
```

itertools (compact combinatorics)

```
from itertools import chain, product, combinations
list(chain(xs, ys))
product("AB", repeat=2); combinations(items, 2)
```

18. Common Python Patterns

```
a, b = b, a                      # swap
first, *middle, last = values    # starred
unpacking
unique = list(dict.fromkeys(items)) # dedupe, keep
order
users.sort(key=lambda u: u["age"]) # custom sort
key
```

```
from collections import Counter
counts = Counter(items)          # frequencies
for i, value in enumerate(items): ...
for a, b in zip(xs, ys): ...
```

```
has_large = any(x > 10 for x in nums)
all_positive = all(x > 0 for x in nums)
flat = [x for row in matrix for x in row]
value = data.get("key", default)
```

```
# group rows by a computed key
groups = defaultdict(list)
for row in rows:
    groups[row.category].append(row)
```

```
# EAFP: perform the normal operation, handle failure
try: value = data[key]
except KeyError: value = default
```

19. Common Gotchas

Gotcha	Preferred practice
<code>a == b</code> vs. <code>a is b</code>	equal values vs. same object; use <code>is</code> <code>None</code>
<code>xs.sort()</code> vs. <code>sorted(xs)</code>	mutates and returns <code>None</code> vs. returns new list
mutable function defaults	use <code>None</code> , then create inside
shallow copy	nested objects remain shared; use <code>copy.deepcopy</code> only when required
float equality	use <code>math.isclose(a,b)</code>
mutating during iteration	iterate over <code>xs.copy()</code> or build a new collection

```
# BAD: rows refer to the same inner list
grid = [[0] * 3] * 3
# GOOD: a distinct list per row
grid = [[0] * 3 for _ in range(3)]
```

```
# Filtering without modifying the traversed list
items = [x for x in items if keep(x)]
```

Also remember: assignment does not copy objects; `list.copy()` and slicing are shallow. Avoid broad `except` Exception unless re-raising or at a deliberate application boundary.